

Chapter 8

Understanding Object Model Diagrams

This chapter will first explain what ArcObjects are and where to find ArcObjects help resources. Then it will introduce a few key Unified Modeling Language (UML) notations for classes and class relationships. While the emphasis will be on the interpretation of ArcObjects model diagrams and navigation between ArcObjects components, a number of Component Object Model (COM) related concepts and programming techniques commonly used in ArcObjects component development will be introduced and discussed with examples. These concepts and techniques include multiple interface, query interface, class types, class inheritance, as well as interface inheritance. Finally, several frequently used ArcObjects assemblies and key components will be listed.

1. About ArcObjects

ArcGIS desktop add-in development introduced in Chapter 7 allows you to create add-in components such as buttons, tools, combo boxes, and toolbars in ArcGIS desktop applications such as ArcMap and ArcCatalog. To make add-in components to perform GIS tasks, you must write code with ArcObjects.

Nowadays, people can build homes using home components such as doors, windows, and even whole walls and roofs. Some college students build their own computers using standard parts including CPUs, motherboards, hard drives, memory chips, and graphics cards obtained from online stores. In these examples, the doors, windows, CPU, and hard drives etc. each is an independently functional component. In manufacturing industry, building products from standardized components greatly increases productivity. In software industry, component-based software engineering which assembles software products from prefabricated reusable components can also minimize cost and shorten release cycle time. In addition, standard software components also facilitates development of custom tools and functions.

Object-oriented programming has made it possible to assemble software products from standardized components. ESRI's ArcGIS products are built from a library of software components called ArcObjects. ArcObjects components, which are similar to home or PC components, are created and used by ESRI to build the ArcGIS applications including ArcMap and ArcCatalog. ESRI distributes not only assembled GIS products but also the foundation components to allow developers to create custom tools and applications in order to satisfy special needs.

The ArcObjects library is made up of thousands of components each having a unique structure and intended purposes. Various components are related to one another to make a complex system, which lends ArcObjects to allusions to a fantastic circuit city in which grotesque structures are symbols of unique components and road-like circuit lines signify relationships between components (Figure 1). To travel in such a fanciful city without getting lost, you must have a tour guide book or an atlas of the city. The tour guide book is the ArcObjects SDK online documentation, particularly, the ArcObjects API Reference currently available at:

<http://resources.arcgis.com/en/help/arcobjects-net/componenthelp/index.html>. The city atlas is

the ArcObjects object model diagrams which consists of dozens of pdf documents normally available in the ArcGIS development kit installation folder. You can also find all the pdf files in the accompanying CD-ROM in folder “Diagrams”.



Figure 1 City of ArcObjects

(Adapted from <http://www.sciencephoto.com/media/429885/enlarge>)

Thousands of ArcObjects components are grouped into 79 assemblies¹. An assembly is a .NET term for a package. An assembly can contain from several to hundreds of components. The ESRI online documentation provides detailed description of each component, which includes properties and methods of the component as well as relationships between components. However, the documentation is a reference for experienced users.

ESRI also provides a set of object model diagrams for each assembly. These diagrams use a popular notation language called Unified Modeling Language (UML) to describe the ArcObjects components, which include their interfaces (properties, methods) and relationships between components. To get started with ArcObjects programming, the rest of this chapter will first introduce some key UML notations, and then discuss several key ArcObjects concepts and techniques as well as their UML representations. You will learn the basic skills of writing code based on interpretation of object model diagrams. As you follow the book into add-in tool development, more and more ArcObjects components will be explored and practiced.

2. UML for Class Diagrams

¹ <http://resources.arcgis.com/en/help/arcobjects-net/componenthelp/index.html>

In order to explore the fantastic City of ArcObjects, you must understand some basic notations of the Unified Modeling Language (UML). UML is a notation language that uses a number of symbols to describe the structure of classes and relationships between them. It is popularly used to represent object-oriented designs, and it has strength in describing complex systems such as the entire ArcGIS software family.

2.1 Class structure

In UML, a class is represented by a box usually consisting of three stacking compartments. The top compartment contains the name of the class; the middle compartment contains the class properties each specified with a data type; and the lower compartment contains class methods. If a method returns a value, a return data type is specified behind. The Point class can be described by UML diagram shown in Figure 2.

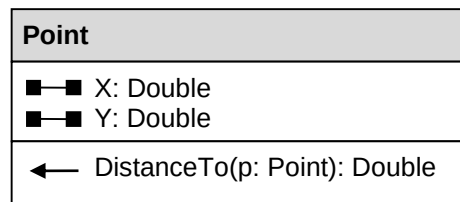


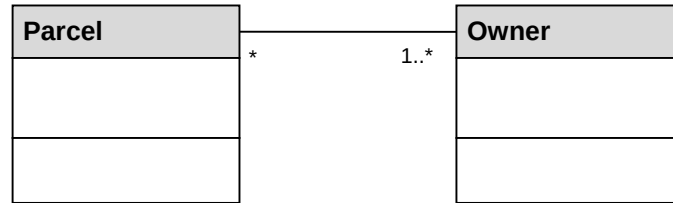
Figure 2 UML notations for the Point class structure

This class has two properties, *X* and *Y*, both are double-precision floating point numbers. It has one method *DistanceTo()* that returns a double-precision floating point number. In ArcObjects model diagrams, a property is labeled with a dumbbell icon. A double ended a dumbbell (■—■) indicates a two-way (read and write) property. A right-ended dumbbell (—■) indicates a write-only property and a left-ended dumbbell (■—) indicates a read-only property. A method is indicated by a little left-pointing arrow pointing outward (←). The data type of a property, the return value of a method, or a parameter of a method is denoted by a colon followed by a data type. In Figure 2, both *X* and *Y* properties are Double data type, the parameter (*p: Point*) of the *DistanceTo()* method is a Point type, and the return value of the method is a double-precision floating point number.

2.2 Class relationships

In object-oriented programming, one object can relate to another in any of these four relationships: *association*, *inheritance*, *aggregation*, and *creation*.

- 1) **Association** defines a general relationship between two classes. Association is denoted by a solid line connecting two classes. Numbers or asterisk symbols (*) are usually placed at the ends of the line to indicate cardinality, i.e. the number of entities participating the relationship (Figure 3).



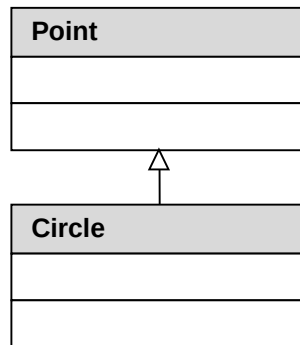
“Parcels are related to owners”

One parcel can be related to one or more owners (1..*)

One owner can be related to any number of parcels (*)

Figure 3 UML notations for association relationship

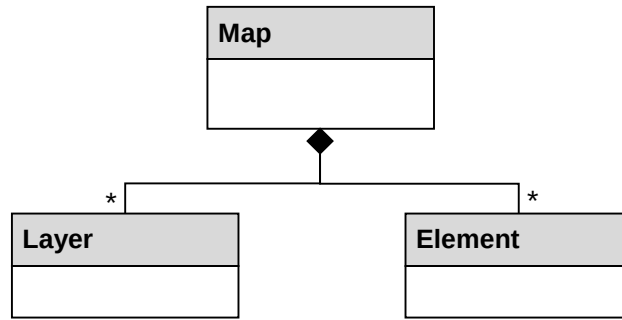
- 2) **Inheritance** defines a parent-child relationship in which the child class inherits all properties and method of the parent class. Conceptually, a child class can be thought of as “a kind of” the parent class. Inheritance is denoted by a solid line connecting two classes with a little triangle attached and pointing to the parent class (Figure 4)



The Circle class inherits the Point class so that it takes on all properties and methods of the Point class.

Figure 4 UML notations for inheritance relationship

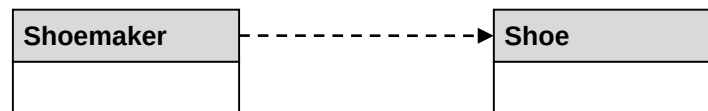
- 3) **Aggregation** defines a whole-part relationship in which one object contains or is composed of other objects. For example, a polyline consists of a number of points, and a map is made up of layers and map elements. Aggregation is denoted by a solid line connecting two classes with a little diamond attached to the end of whole side (Figure 5)



A map is composed of layers and elements.
The Map class represents the whole and the Layer and Element classes are parts.

Figure 5 UML notations for aggregation relationship

- 4) **Creation** refers to the relationship in which one object produces another object. In object-oriented programming, an object may be created by another object of a different class. For example, a shoemaker object can produce a shoe object and a chicken can lay eggs. A creation relationship is denoted by a dashed line with an arrow pointing to the resulting object (Figure 6). This will be further explained in section 3.3 c of this chapter.



A shoemaker makes shoes

Figure 6 UML notations for instantiation relationship

Once you know these UML notations, you can begin to read ArcObjects model diagrams.

3. Concepts and techniques of ArcObjects programming

3.1 Component Object Model (COM)

All ArcObjects components are classes developed to the Microsoft Component Object Model (COM) standards and are enabled for .NET programming. The COM specifications support the separation of object interface and implementation. This means that when a COM component is created, the declarations of interfaces (including properties and methods) of a class can be separated from the code that implements those interfaces. As such, the interface declarations and implementation code may be stored in separate files. Please recall the Point class you created in Chapter 6. The declaration of interface (the X and Y properties and the DistanceTo method) and implementation of the interface are lumped in one file named Point.vb. With separation of interface and implementation in COM, the interface of a class can stay unchanged during a version upgrading, while the code that implements the class methods may be overhauled. As a result, the internal changes have minimal impacts on application development because component interfaces are the same. For example, When ArcObjects upgraded from version 10.1 to version 10.2, the interfaces of the Map class was not changed, so that we see

the same properties (such as Name, MapScale ...) and methods (such as AddLayer, SelectFeature ..), although the code or algorithms that implement some of the methods might have been substantially changed for performance enhancement. As application developers, we do not have to be concerned about the internal operation of a method in a class. As long as interfaces are the same, we can continue to use the components as if nothing has changed after software upgrading. This is similar to the upgrading of car models. We do not have to understand all the details under the hood, as long as the steering wheel, the shift stick, and the pedals are the same, we can drive the new car without additional training.

3.2 Multiple interfaces

Another advantage of the separation of class interface and implementation is that it helps organize interfaces. GIS components are often complicated due to the fact that a component usually has many properties and methods. For example, the Map component (class) has hundreds properties and methods although some of them are rarely used. When ESRI creates the Map class, it categorizes the numerous properties and methods into 21 interface files, so that the Map class has 21 interfaces. Having multiple interfaces is a nature of ArcObjects components.

An interface is a group of relevant properties and methods. To distinguish different interfaces of a class, each interface is assigned a name. By convention, interface names are prefixed with letter "I". For example, the Map class has interfaces "IMap", "IBasicMap", "IGraphicsContainer", "IActiveView" and so on. Please open the "CartoObjectModel.pdf" document and take a look at all interfaces of the Map class. Zoom in to the document to see details.

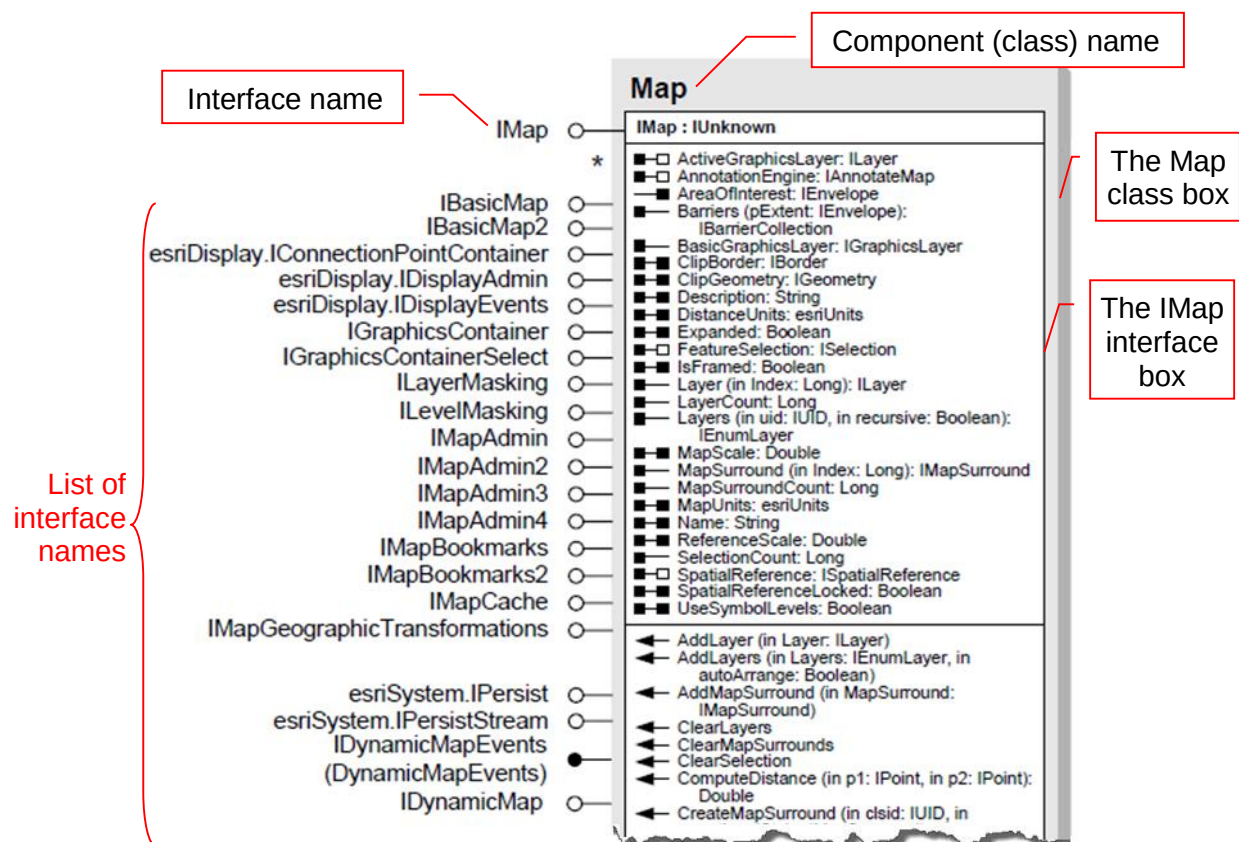


Figure 7 The IMap interface of the Map class

In ArcObjects model diagrams, an interface is usually denoted by three stacked boxes with a handle resembling a lollipop. The top box contains the interface name, the second box holds properties, and the bottom box contains methods (Figure 7).

Figure 7 only displays a small portion of the Map class. In particular, it only displays a part of the IMap interface of the Map component (class). The overall 3-D box in gray represents the Map class, and the white boxes inside the class box denotes the IMap interface. While the IMap lollipop symbol is connected to the inner IMap interface box, a long list of lollipops are only connected to the overall Map class box. Because of limited page space, Figure 7 only displays part of the contents of the IMap interface, and lists names of many other interfaces without showing their contents. In case you want to use a property or method on an interface in the list, you have to find the diagram or description of the interface somewhere else in the complete PDF document, other ArcObjects diagrams, or the ArcObjects SDK online documentation.

3.3 Query Interface (QI)

When you instantiate a new object from a class that has multiple interfaces, you must specify an interface to use. For example, when you create a new map object from the Map class, you may use the following code.

```
Dim map As IMap = New Map()  
Or  
Dim av As IActiveView = New Map()
```

The first line creates a new map object and stores its address in new variable *map* which points to the IMap interface, so that you can use properties and methods on the IMap interface through variable *map*. The second line creates a new map object and stores its address in new variable *av* which points to the IActiveView interface, so that you can use properties and methods on the IActiveView interface through variable *av*. For example, if you want to refresh the map, you need to use the *av* variable because the Refresh() method resides on the IActiveView interface (Figure 8). If you want to add a layer into the map, you must use the *map* variable because the AddLayer() methods is on the IMap interface. I suggest you to take a look at the two interfaces in the “CartoObjectModel.pdf” PDF document to find these two methods.

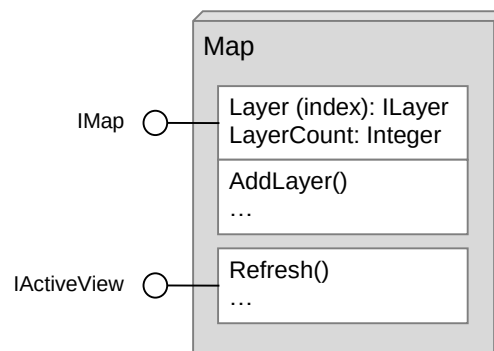


Figure 8 Two interfaces of the Map class

IUnknown

- ActiveGraphicsLayer: ILayer
- AnnotationEngine: IAnnotateMap
- AreaOfInterest: IEnvelope
- Barriers (pExtent: IEnvelope):
 - BarrierCollection
- BasicGraphicsLayer: IGraphicsLayer
- ClipBorder: IBorder
- ClipGeometry: IGeometry
- Description: String
- DistanceUnits: esriUnits
- Expanded: Boolean
- FeatureSelection: ISelection
- MoveLayer (in Layer: ILayer, in toIndex: Long)
- RecalcFullExtent
- SelectByShape (in Shape: IGeometry, in env: ISelectionEnvironment, in justOne: Boolean)
- SelectFeature (in Layer: ILayer, in Feature: IFeature)
- SetPageSize (in widthInches: Double, in heightInches: Double)

IActiveGraphicsLayer

- ActiveGraphicsLayer: ILayer
- AnnotationEngine: IAnnotateMap
- AreaOfInterest: IEnvelope
- Barriers (pExtent: IEnvelope):
 - BarrierCollection
- BasicGraphicsLayer: IGraphicsLayer
- ClipBorder: IBorder
- ClipGeometry: IGeometry
- Description: String
- DistanceUnits: esriUnits
- Expanded: Boolean
- FeatureSelection: ISelection
- MoveLayer (in Layer: ILayer, in toIndex: Long)
- RecalcFullExtent
- SelectByShape (in Shape: IGeometry, in env: ISelectionEnvironment, in justOne: Boolean)
- SelectFeature (in Layer: ILayer, in Feature: IFeature)
- SetPageSize (in widthInches: Double, in heightInches: Double)

So in programming, suppose we have an object represented by a variable pointing to one interface, but we need to use a property or method residing on another interface of the same object, do we have to instantiate another new object from the same class like we did above? For example, if we only have one map object represented by variable *map* which points to the IMap interface, however, we want to use the Refresh() method which resides on the IActiveView interface, do we really have to create another new map object and store its address in another variable (like *av*) that points to the IActiveView interface? The answer is “No”.

```
Dim av As IActiveView ' 1) declares a variable pointing to the target interface
av = map               ' 2) assigns the existing object to the new variable
```

```
map.AddLayer(<a layer>)
Av.Refresh()
```

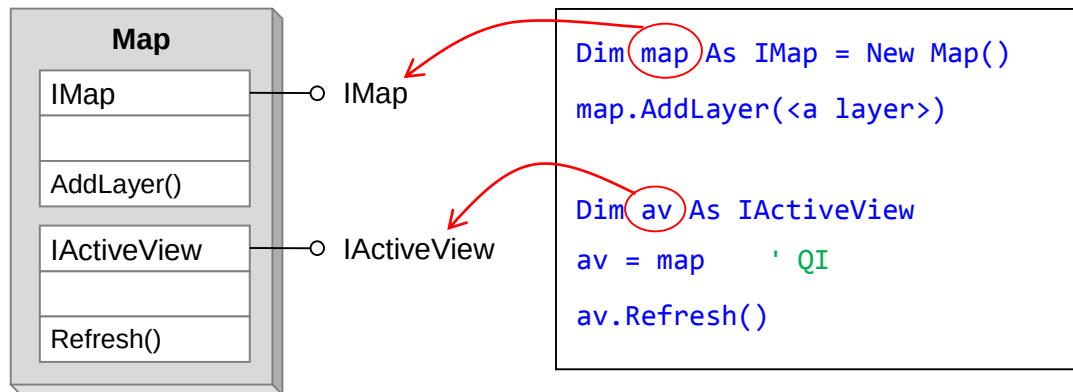



Figure 10 Query Interface

In this example, the QI code line (`av = MyMap`) assigns one interface of an object to a variable pointing to another interface. Unlike many other languages, such as C++ and Java, which are very strict in data type consistence while assigning values, VB.NET is much more liberal in dealing with data types. For example, you may sometimes assign a string value to a numeric variable in VB and the language takes care type conversion automatically. However, if you add statement “Option Strict On” at the top of a code module, VB can become as harsh as C++ on data types. If this is the case, an explicit data type conversion is required in query interface, e.g.

```
av = TryCast(map, IActiveView)
```

Where `TryCast()` is a VB.NET built-in function that casts (converts) an object from one interface into another. The `TryCast` function takes two parameters: an input object (e.g. `map`) and a target Interface (e.g. `IActiveView`). If casting is not possible, for example, to cast a `map` object with `IMap` interface (e.g. `map`) to the `ILayer` interface of the `Layer` class (a different class), the result will be `Nothing` (in VB, `Nothing` represents the default value of any data type) and the code does not crash.

3.4 Class types

The Microsoft COM model supports three different types of classes: *co-class*, *abstract class*, and *regular class*.

1) Co-class

A *co-class* is a class that can be used to instantiate objects by using keyword “New”. The `ArcObjects` component `Map` is a co-class so that we can create new `map` objects from it. For instance,

```
Dim map As IMap = New Map()
```

In `ArcObject` model diagrams, a co-class is denoted by a 3-D gray box. Figure 11 shows that `Point` is a co-class, so we can instantiate `point` objects from it.

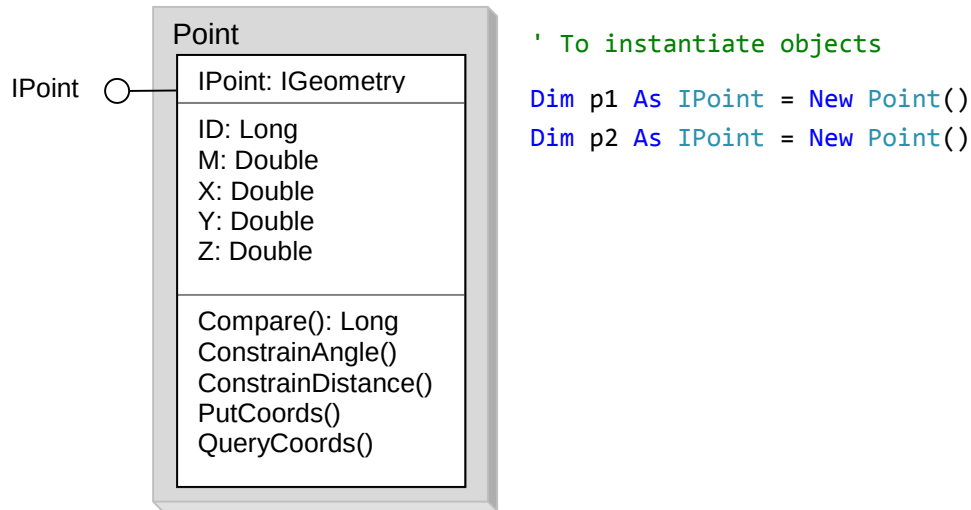


Figure 11 Object model diagram for co-class Point

2) Abstract class

An *abstract class* is a class that cannot instantiate objects. In ArcObjects model diagram, an abstract class is denoted by a 2-D gray box. In Figure 12, the Geometry class is an abstract class. Technically, an abstract class only provides interfaces for its child co-classes to inherit, and the class itself does not contain code that implements the interfaces. Given Figure 12, we cannot create a geometry object using code

Wrong: `Dim aGeometry As IGeometry = New Geometry()`

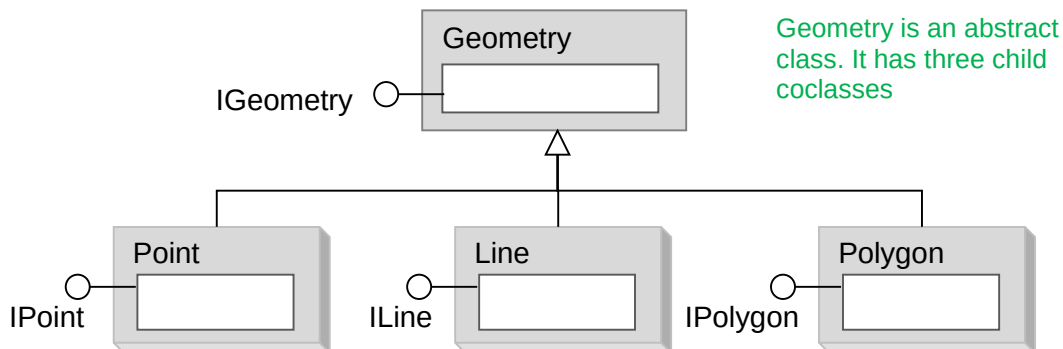


Figure 12 An abstract class is denoted by a 2-D gray box
An abstract class is usually inherited by child coclasses

An abstract class must be inherited by some child co-classes. Figure 12 shows that the Geometry class is inherited by three child co-classes. If we need an interface on an abstract class, we can instantiate an object from an appropriate child co-class and perform a QI to switch to the target interface. For example, we can instantiate a point object from the Point co-classes.

```
Dim p As IPoint = New Point()
```

If an IGeometry interface is needed, we can perform QI to switch to that interface.

```
Dim geom As IGeometry = p ' QI to the IGeometry interface
```

The two lines of code above may be combined into one as the following line. But please be aware of the implied Query Interface below.

```
Dim geom As IGeometry = New Point() ' QI implied
```

After switching to the IGeometry interface by QI, we can use any property or method on the IGeometry interface via variable *geom*.

3) Regular class

A *regular class* also cannot instantiate objects from itself. In ArcObjects model diagrams, a regular class is denoted by a 3-D white box. If class Shoe is a regular class, we cannot create a shoe object by using the following code:

```
Wrong: Dim shoe As IShoe = New Shoe
```

Objects of a regular class must be created by an object of a different class. In order to create an object of a regular class, we have to find an object of another class that has a method for creating the desired object. For example, if class ShoeMaker has a method named CreateShoe() on interface IShoeMaker, then you can use a shoe maker object to make a shoe object by calling that method. Suppose we have a shoe maker object represented by variable *shoemkr* pointing to the IShoeMaker interface, the following line creates a shoe object.

```
Dim shoe as IShoe = shoemkr.CreateShoe()
```

Figure 13 shows ArcObjects components FeatureClass and Workspace are regular classes so that they cannot instantiate objects by themselves. However, feature class objects can be created by a workspace object because the IFeatureWorkspace interface has a method named “OpenFeatureClass” that returns a feature class object (with IFeatureClass interface, denoted by OpenFeatureClass(name): IFeatureClass). To obtain a workspace object, we must use a workspace factory object since the WorkspaceFactory class has a method OpenFromFile() able to create a workspace object (with IWorkspace interface).

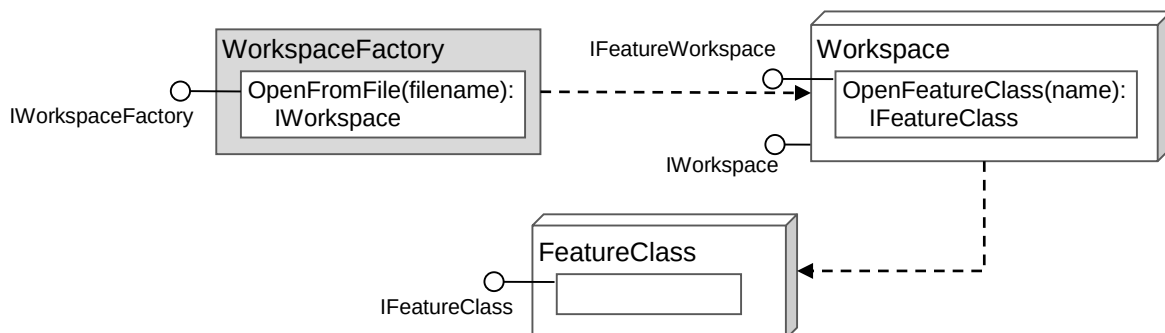


Figure 13 Regular classes are denoted by 3-D white boxes
Objects of regular classes must be created by objects of other class

Given a workspace factory object represented by variable *wksFactory* which points to the *IWorkspaceFactory* interface, we can call its *OpenFromFile()* method to create a workspace object. For example,

```
Dim wks As IWorkspace = wksFactory.OpenFromFile(<arguments...>)
```

Now a workspace object is created and represented by variable *wks* that points to the *IWorkspace* interface. We can use this object to create a feature class object by calling the *OpenFeatureClass()* method on the *IFeatureWorkspace* interface. This task can be carried out in two steps as the following:

```
' first, perform a QI to switch to the IFeatureWorkspace interface
Dim fwks As IFeatureWorkspace = wks

' second, call the OpenFeatureClass() method to create a feature class object
Dim fclass As IFeatureClass = fwks.OpenFeatureClass("Parcel")
```

The resulting feature class object is now represented by variable *fclass* which points to the *IFeatureClass* interface.

3.5 Class inheritance and interface inheritance

As described in Section 2.2, class inheritance is denoted by a solid line ended with a little triangle pointing to the parent class. Class inheritance means a class inherits all interfaces of its parent class. In Figure 14, the *FeatureLayer* co-class inherits the *Layer* abstract class. In this example, the *FeatureLayer* class inherits (can use) all interfaces of the *Layer* class. As a result, we may think that the *FeatureLayer* class also includes the *ILayer* interface along with other interfaces of its own (see right part of Figure 14).

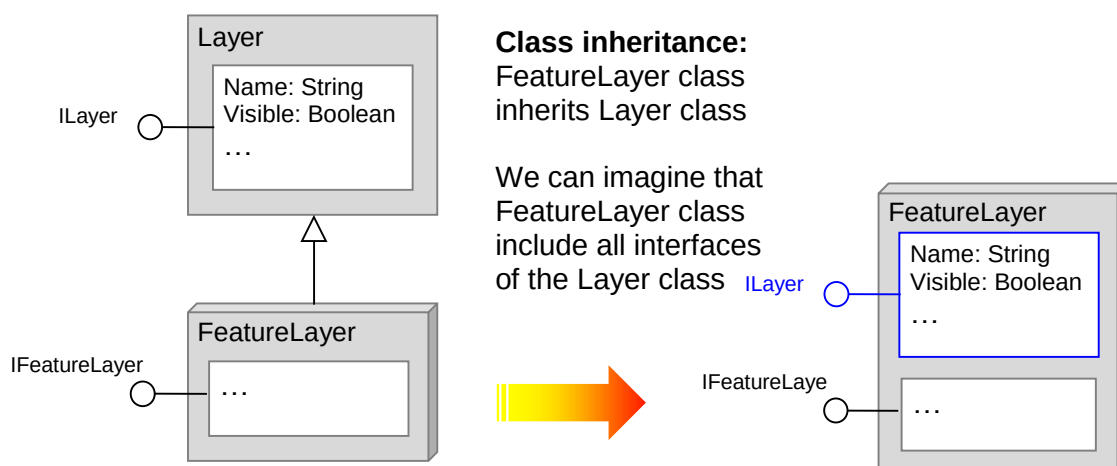


Figure 14 Class inheritance: a child class inherits all interfaces of its parent class

Interface inheritance is denoted in the interface title compartment (the top box of an interface) by a colon followed by the parent interface name. In Figure 15, the *ILineSymbol* interface inherits *IUnknown* interface (denoted by *ILineSymbol: IUnknown*), and the *ISimpleLineSymbol* inherits the *ILineSymbol* (denoted by *ISimpleLineSymbol: ILineSymbol*).

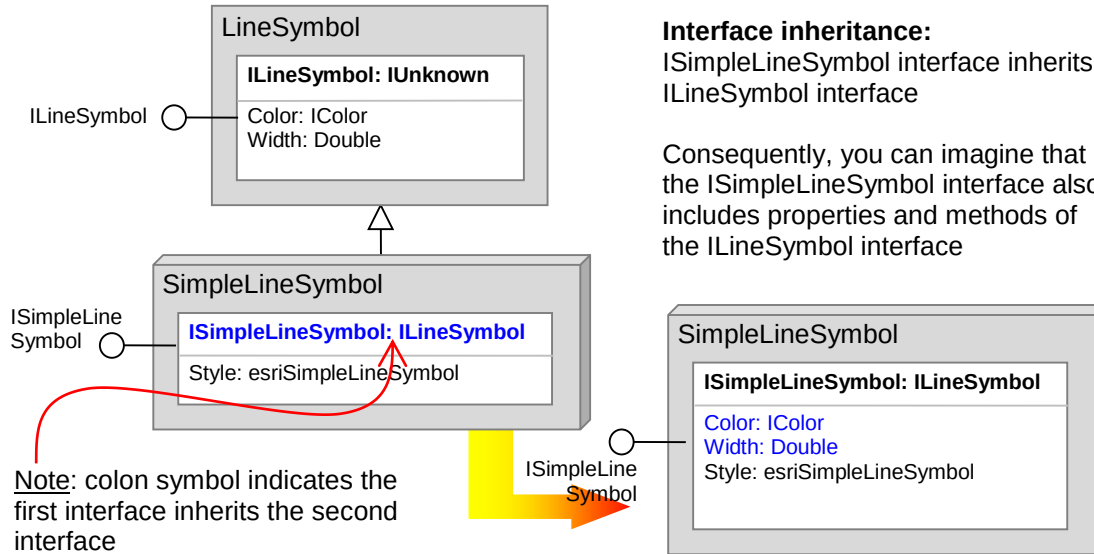


Figure 15 Interface inheritance: a child interface inherits all contents of its parent interface

With interface inheritance, the child interface inherits all the properties and methods of the parent interface. In Figure 15, since the ISimpleLineSymbol interface inherits the ILineSymbol interface, all properties (Color and Width properties in diagram) on the ILineSymbol are inherited by the ISimpleLineSymbol interface. Therefore, we can think that the property compartment of the child interface also includes all the inherited properties along with its own properties (see right part of Figure 15). And the method compartment of the child interface also includes all inherited methods in addition to its own methods. If you create a simple line symbol object and use a delegate variable pointing to the ISimpleLineSymbol interface, you can directly access the color and width through that variable without using QI. For example:

```
Dim sym As ISimpleLineSymbol = New SimpleLineSymbol()
sym.Width = 2 ' variable sym can directly access the Width property
```

Okay, that's all about the ArcObjects model diagram language. Once you learn this language, you are ready to set off for the fantastic City of ArcObjects. You will use the ArcObjects model diagrams in all subsequent weeks.

4. ArcObjects Assemblies and Namespaces

Given the gigantic size of the City of ArcObjects with complicated roads, you may still get lost even though you have a map in hand. Before you enter the city, let me quickly show you the general layout of the city and some important landmarks.

In ArcObjects, components are organized in assemblies (or packages) resembling districts of a city, such as central business district, industrial parks, residential neighborhoods, etc. Each assembly has a unique namespace. Since namespaces are often organized in a hierarchical structure, the full namespace of an ArcObjects assembly is usually a series of namespaces separated by dot symbols (.), for instance, ESRI.ArcGIS.Carto, ESRI.ArcGIS.Display, and

ESRI.ArcGIS.Geometry. You can find reference to all the 79 namespaces of ArcObjects including thousands of components at: <http://resources.arcgis.com/en/help/arcobjects-net/componenthelp/index.html>

When you need a particular ArcObjects component or GIS function, think about which namespace it may logically fall in. You may use the search tool on the online documentation page to search keywords. To become efficient in finding necessary components, you should get familiar with some frequently used assemblies and namespaces. The following introduces some common namespaces (assemblies) and frequently used components:

ESRI.ArcGIS.ArcMap: The ArcMap assembly includes an important object *Application* with two important interfaces *IApplication* and *IMxApplication*. *Application* is the overall object of the ArcGIS desktop application in which a program runs. If an add-in program runs in ArcMap, the application object is the ArcMap window. This assembly is not automatically loaded into an add-in project by the Add-in Wizard. If you need to use the application object, you must manually load it the assembly as a reference into your project. Adding assemblies into a VB project will be introduced in Chapter 9.

ESRI.ArcGIS.ArcMapUI: The ArcMapUI assembly provides user interface (UI) components specific to the ArcMap application. This assembly contains an important component *Document* which has a highly accessed interface *IMxDocument*. The *IMxDocument* interface provides access to the map document (the .mxd file) object loaded in the ArcMap application. In ArcMap add-in programming, the application object and document object are automatically instantiated in project namespace *My.ArcMap* as soon as the application is started. Therefore, the application object with *IApplication* interface can be accessed by *My.ArcMap.Application*, and the document object with *IMxDocument* interface can be accessed by *My.ArcMap.Document*. No customer code is needed to instantiate these two objects. This assembly is automatically loaded into new add-in projects by the Add-in Wizard.

ESRI.ArcGIS.Carto: The Carto assembly provides components for manipulating map elements and cartographic design. Frequently used classes include *Map*, *Layer*, *PageLayout*, *Element* (such as *GraphicElement* and *FrameElement*), *MapSurround* (including *Legend*, *MapInset*, *Scalebar*, etc.), *Renderers* (including *SimpleRenderer*, *UniqueValueRenderer*, *ClassBreaksRenderer*, *DotDensityRenderer*, etc.), components for labeling and annotation, and so on. This is a very important assembly because as long as your project needs to access map layers, you must refer to this assembly. You will encounter this assembly in every project when you continue to follow this book. This assembly is not preloaded into a new add-in project by the Add-in Wizard, so you are suggested to add this assembly into your new add-in projects as soon as they are created.

ESRI.ArcGIS.Display: The Display assembly is also a fundamental library. The Carto assembly has a dependency on it when manipulating map layers. Frequently used classes in this assembly include *Color*, *Symbol*, *RubberBand*, *Tracker*, *ScreenDisplay*, and *DisplayTransformation*. You will first encounter this assembly in Chapter 10, and will need to refer to this assembly when you work with color or symbol objects while rendering maps. This assembly is not preloaded by the Add-in Wizard.

ESRI.ArcGIS.Geodatabase: The Geodatabase assembly provides components to access geospatial data. Common components in this assembly include *Workspace*, *FeatureDataset*, *FeatureClass*, *RelationshipClass*, *Dataset*, *Geodataset*, *Table*, *Row*, *ObjectClass*, *Object*, and so on. You will first encounter this assembly in Chapter 10 and will need to refer to this

assembly when you work with spatial or attribute data of feature classes. This assembly is not preloaded into new add-in projects by the Add-in Wizard.

ESRI.ArcGIS.Geometry: The Geometry assembly provides components for manipulating ESRI supported geometric primitives. Frequently used components in this assembly include *Points*, *Polylines*, *Polygons*, *Multipoints*, *MultiPatches*, *Paths*, *Rings*, *Segments*, etc. You will begin to use this assembly from Chapter 11 and often need to refer to this assembly whenever you want to access spatial data in feature classes or want to create new geometric primitives. This assembly is not preloaded into new add-in projects by the Add-in Wizard.

ESRI.ArcGIS.Geoprocessing: ArcGIS includes over 400 such geoprocessing tools and operators for manipulating and analyzing geographic information. To make use of these ArcGIS built-in tools as well as customer provided tools in add-in environment, the Geoprocessing assembly provides an important component *Geoprocessor* that can sets geoprocessing environment and executes geoprocessing tools. Using geoprocessing tools in add-in programming will be introduced in Chapter 15. The Geoprocessing assembly is not preloaded into new add-in projects by the Add-in Wizard.

ESRI.ArcGIS.Framework: The Framework assembly is preloaded into new add-in projects by the Add-in Wizard. It provides fundamental components including *Application*, *Document*, *Dockable windows*, *Command bars*, *Commands*, and so on.

ESRI.ArcGIS.System: The System assembly is the lowest-level assembly in the ArcGIS architecture. This assembly contains components that expose services used by other assemblies comprising ArcGIS. Examples of components included in this assembly include *Extension*, *Array*, *Set*, *Stream*, *Classify*, and *NumberFormat*. This assembly is preloaded into new add-in projects by the Add-in Wizard.

Exercises

1. Answer the following questions.
 - 1) What is ArcObjects?
 - 2) Who created ArcObjects components?
 - 3) What is an assembly?
 - 4) What does UML stands for?
 - 5) Is UML a programming language? If not, what is it?
 - 6) How is a class described in UML?
 - 7) How are properties of a class described in UML?
 - 8) How are methods of a class described in UML?
 - 9) How is an association relationship denoted in UML?
 - 10) In object-oriented modeling, what does inheritance mean?
 - 11) How is an inheritance relationship denoted in UML?
 - 12) In object-oriented modeling, what does aggregation mean?
 - 13) How is an aggregation relationship denoted in UML?
 - 14) What is an interface of a class (or object)?
 - 15) Why multiple interfaces are used in ArcObjects components?
 - 16) What is Query Interface (QI)? Why do we need QI?
 - 17) How do you perform query interface in programming?
 - 18) What does function TryCast() do?
 - 19) What is a co-class? In ArcObject model diagram, how is a co-class described?
 - 20) What is an abstract class? How is an abstract class described In ArcObject model diagram?
 - 21) What is a regular class? How is a regular class described in ArcObject model diagram?
 - 22) What happens if a class inherits another class?
 - 23) What happens if an interface inherits another interface?
 - 24) What is a namespace? Can you name a few ArcObjects namespace?

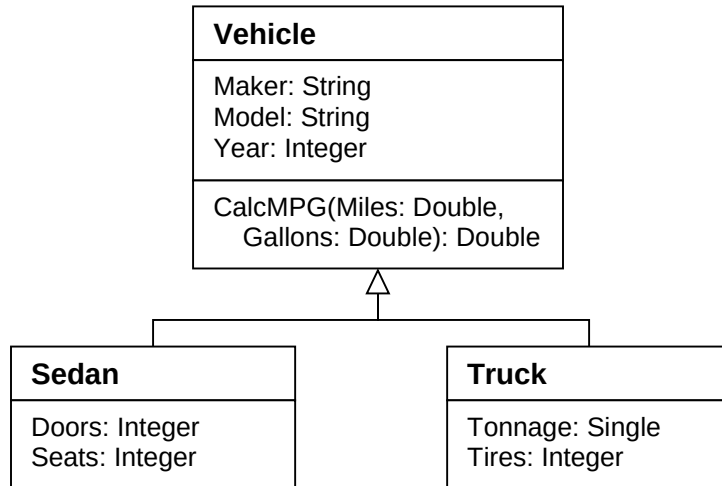
Explain the following UML diagrams in questions 2-6.

2.

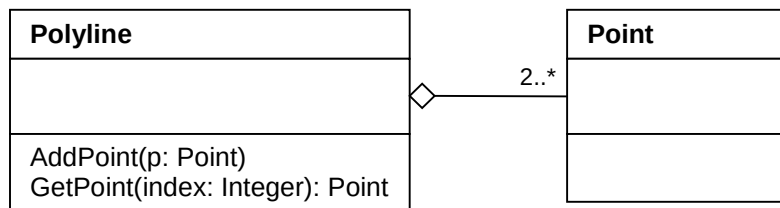
Point
X: Double Y: Double Z: Double
DistanceTo(p: Point): Double DistanceTo(x: Double, y: Double, z: Double): Double

Polyline
Points(): Point Length: Double PointCount: Integer
AddPoint(p: Point) RemovePoint(index: Integer)

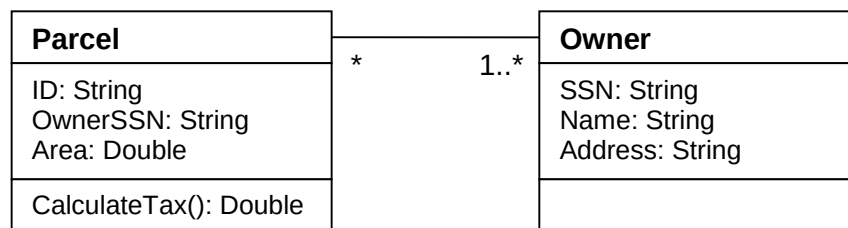
3.



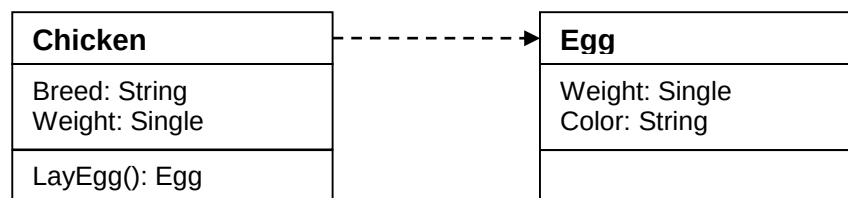
4.



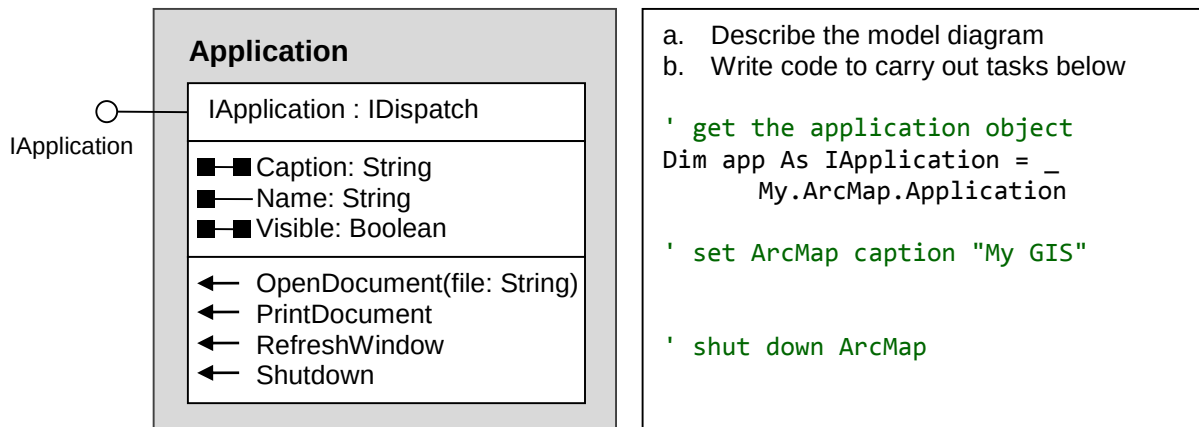
5.



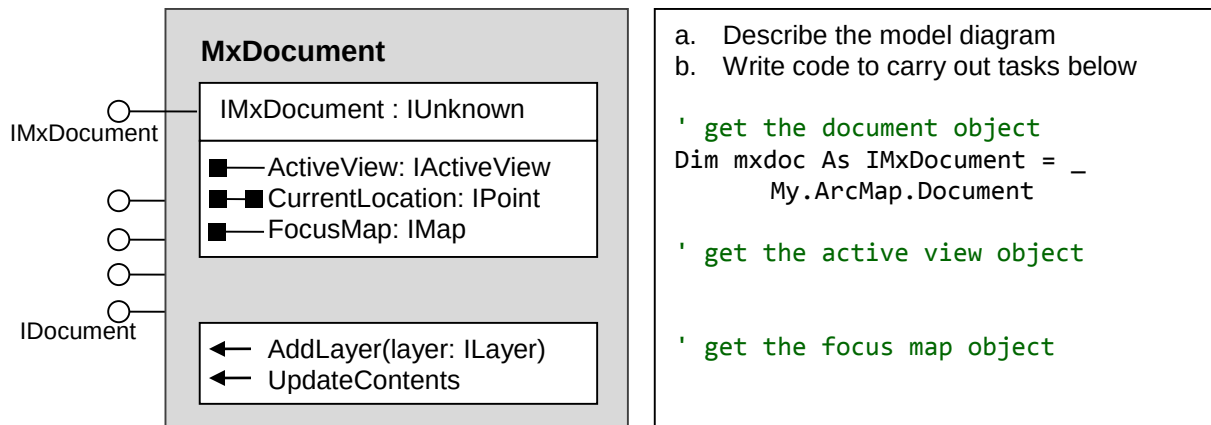
6.



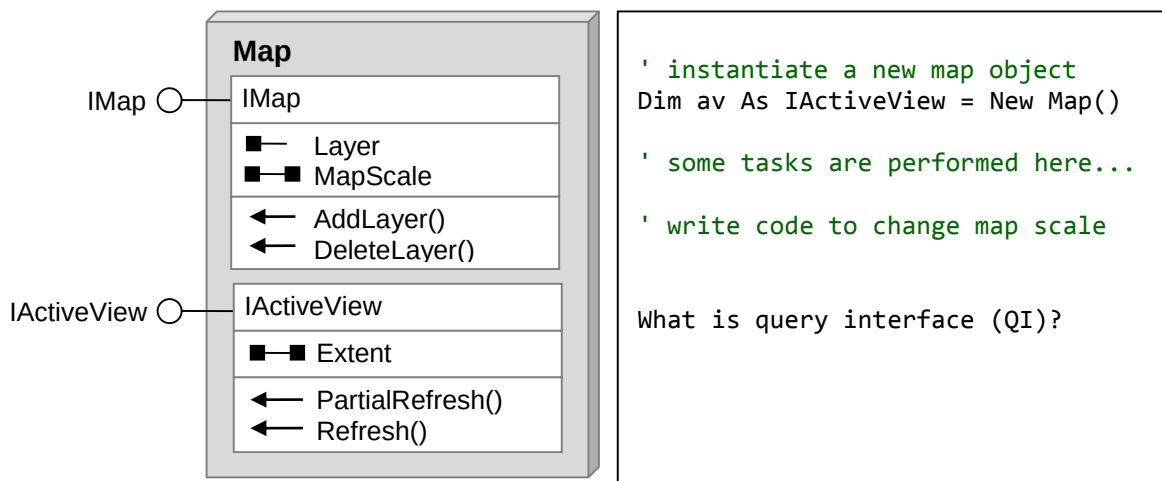
7. Analyze the Application object model diagram and complete code in the box.



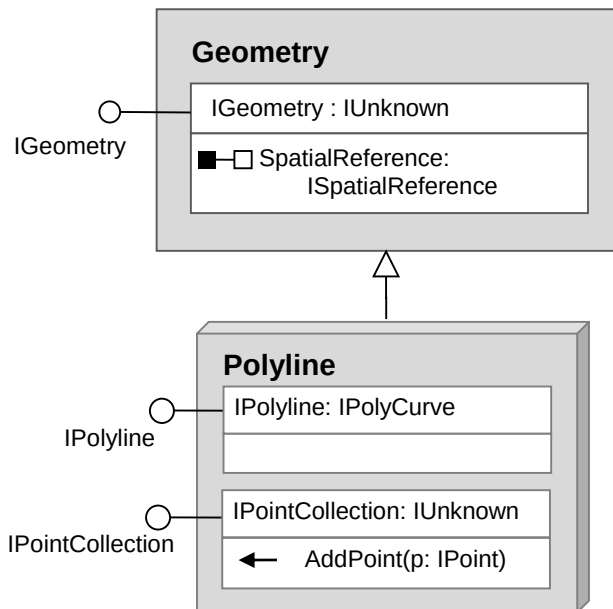
8. Analyze the MxDocument object model diagram and complete code in the box.



9. Analyze the Map object model diagram, complete code and answer the question in the box



10. Analyze the Geometry object model diagram, complete code and answer questions in the box



a. Instantiate a polyline object
`Dim pline As IPolyline = New Polyline`

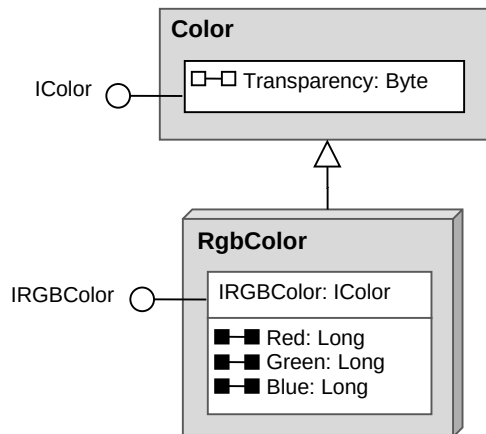
b. Add a point object p into to the
polyline object

c. Set a spatial reference object sr
to the polyline object

d. How is class inheritance denoted?

e. What happens if one class
inherits another class?

11. Color components



- Instantiate an `RgbColor` object and set the following properties:
Red = 255
Green = 127
Blue = 127
- Set the `Transparency` property of the color object with value 255 (opaque)

How is interface inheritance denoted?

What happens if one interface inherits another interface?

12. Explain the following ArcObjects model diagram.

